```
Proposal for C23
WG14 xxxx

Title:                Type inference for object definitions
Author, affiliation: Jens Gustedt, INRIA/ICube, Universite de Strasbourg
                     Alex Gilding, Perforce
Date:                 2022-02-05
Proposal category:    New features
Target audience:      Compiler/tooling developers, application developers
```

Abstract

We propose the inclusion of the so-called `auto` feature for variable definitions into C.

This feature allows declarations to infer types from the expressions that are used as their initializers. This is part of a series of papers for the improvement of type-generic programming in C that has been introduced in N2890.

This takes advantage of the successful integration of the `typeof` feature into C23 and provides basic starting compatibility with the corresponding C++ feature, limited to matching existing practice in shipped C compilers.

Type inference for object definitions

Reply-to: Jens Gustedt (jens.gustedt@inria.fr)
Alex Gilding (agilding@perforce.com)
Document No: Nxxxx
Revises Document No: N2923
Date: 2022-02-05

Summary of Changes

Nxxxx (v7/R6)

- remove type inference for function return types and function parameters
- refocus type inference for objects to exactly match existing C practice and reword in terms of accepted `typeof`

N2923 (v6/R5)

- move handling of auto parameters to N2924
- make type declarations in underspecified declarations undefined behaviour
- add Joseph Myers's wording for the `typeof` specifier, but make it its own sub-phrase
- address an ambiguity concerning the scope of `auto` variables
- make clear that shadowing by `auto` only concerns ordinary identifiers
- minor text adaptions

N2891 (v5/R4)

- straighten the proposed text and move discussion of all special cases (bit-fields, compatible types, VM) into non-normative text such as notes and examples

N2735 (v4/R3)

- some word smithing

N2697 (v3/R2)

- remove a requirement on qualifiers and atomic derivation that was too restrictive

N2674 (v2/R1)

- changes the rules for type inference to use a typeof specifier. This simplifies the rules and brings the defined semantics in line with C++.

N2632

- original proposal

Introduction

Defining a variable in C requires the user to name a type. However when the definition includes an initializer, it makes sense to derive this type directly from the type of the expression used to initialize the variable.

This feature has existed in C++ since C++11, and is implemented in GCC, Clang, and other GNU C compatible compilers using the `__auto_type` extension keyword.

`__auto_type` is a much more limited feature than C++ `auto`, the latter of which is built on top of template type deduction rules. We propose to standardize the existing C extension practice directly. Any valid C construct using this syntax will also be valid and hold the same meaning within the broader semantics of the C++ feature.

Rationale

In N2890 it is argued that the features presented in this paper are useful in a more general context, namely for the combination with lambdas. We will not repeat this argumentation here, but try to motivate the introduction of the `auto` feature as a stand-alone addition to C.

In accordance with C's syntax for declarations and in extension of its semantics, C++ has a feature that allows to infer the type of a variable from its initializer expression.

```
auto y = cos ( x ) ;
```

This eases the use of type-generic functions because now the return value and type can be captured in an auxiliary variable, without necessarily having the type of the argument, here `x`, at hand. That feature is not only interesting because of the obvious convenience for programmers who are perhaps too lazy to lookup the type of `x`. It can help to avoid code maintenance problems: if `x` is a function parameter for which potentially the type may be adjusted during the lifecycle of the program (say from `float` to `double`), all dependent auxiliary variables within the function are automatically updated to the new type.

This can even be used if the return type of a type-generic function is just an aggregation of several values for which the type itself is just an uninteresting artefact:

```
#define div (X , Y )           \
  _Generix (( X ) +( Y ) ,     \
          int : div ,          \
          long : ldiv ,        \
          long long : lldiv )  \
          (( X ) , ( Y ) )

// int , long or long long ?
auto res = div (38484848448 , 448484844) ;
auto a = b * res . quot + res . Rem ;
```

An important restriction for the coding of type-generic macros in current C is the impossibility to declare local variables of a type that is dependent on the type(s) of the macro argument(s). Therefore, such macros often need arguments that provide the types for which the macro was evaluated. This not only inconvenient for the user of such macros but also an important source of errors. If the user chooses the wrong type, implicit conversions can impede on the correctness of the macro call.

For type-generic macros that declare local variables, `auto` can easily remove the need for the specification of the base types of the macro arguments:

```
#define dataCondStoreTG (P , E , D )                \
  do {                                              \
    auto * _pr_p = ( P ) ;                          \
    auto _pr_expected = ( E ) ;                     \
    auto _pr_desired = ( D ) ;                      \
    bool _pr_c ;                                    \
    do {                                            \
      mtx_lock (& _pr_p - > mtx ) ;                 \
      _pr_c = ( _pr_p - > data == _pr_expected ) ;  \
      if ( _pr_c ) _pr_p - > data = _pr_desired ;   \
      mtx_unlock (& _pr_p - > mtx ) ;               \
    } while (! _pr_c ) ;                            \
  } while ( false )
```

C's declaration syntax currently already allows to omit the type in a variable definition, as long as the variable is initialized and a storage initializer (such as `auto` or `static`) disambiguates the construct from an assignment. In previous versions of C the interpretation of such a definition had been `int`; since C11 this is a constraint violation. We will propose to partially align C with C++, here, and to change this such that the type of the variable is inferred from the type of the initializer expression.

We achieve this by standardizing the existing practice in the GNU C dialect provided by the `__auto_type` specifier exactly. This is a strict subset of allowed C++11 behaviour. We expect and hope that implementers will treat incompatibilities with extended C++ declaration syntax (such as `auto const *`) as QoI bugs and implement these as extensions, establishing practice and experience for the C-side interpretation of such declarations. Standardizing the base value-only feature is a necessary basis to allow implementations to build beyond it with extended declarations.

Proposal

We propose standardizing the GNU C feature exactly, except for changing the name of the extended specifier `__auto_type` to the existing specifier `auto`.

The GNU C feature has clear semantics which can be expressed entirely in terms of the adopted `typeof` feature: the inferred type for a given initializer `(init)` is, exactly, `remove_quals (typeof (init))`. For example:

```
void foo (int x, int const y) {
  __auto_type a = x;
  __auto_type b = y;

  int * c = &a;
  int * d = &b; // OK
}

void bar (int x, int const y) {
  typeof (x) a = x;
  typeof (y) b = y;

  int * c = &a;
  int * d = &b; // not OK, qualifier discarded, GCC warns/errors
}
```

The feature is more limited than generic C declarations and than the corresponding C++ feature. A declaration using __auto_type must be initialized, must only consist of a single declarator, and may not have any part of its type specified at all – it must infer the entire object type from the initializer value, and cannot therefore be used in combination with `*` or `[]` the way auto can in C++ - there must be no type specifiers in the sequence apart from the `auto`.

The `auto` used as a complete type specifier may still be used in conjunction with qualifiers and with other storage class specifiers:

```
void baz (int x, int const y) {
  __auto_type const a = x;
  __auto_type b = y;
  static __auto_type c = 1ul; // OK

  int * pa = &a; // not OK
  int const * pb = &b; // OK

  int * pc = &c; // not OK, incompatible with unsigned long *
}
```

In order to avoid either specifying the wording for “same type”, which caused difficulty in accepting revision 5 of this proposal, or allowing different variables with the same syntactic specifier to infer different object types, we propose adding a new syntax constraint to exactly match the current GNU C behaviour that allows only one declarator per whole declaration using `__auto_type`.

We expect implementations to gradually establish practice for how this rule should be relaxed as users explore the design space. As with the restriction against partially-specified types, we hope that implementations will support a more complete, C++-like extended feature that builds on current practice as users start to demand it, but do not standardize invention.

Alternatives

C has accepted final wording for `typeof` and it is therefore now possible to declare an object in terms of the type of its initializer by writing:

```
remove_quals (typeof ((0, init))) var = init;
```

The main problem with this is the repetition. There is a maintenance burden to any use outside of macro expansion; the side effect repetition is potentially unclear; and readability is harmed for non-trivial expressions (which may be quite long).

Practice shows that implementations do not need this repetition. They already know what the type of an initializing expression is, and are able to insert it into the specifiers implicitly. Therefore, the language should not force the use of a repeating construct when one is not necessary.

Impact

Implementation burden for this feature is low. Conforming C implementations are already able to delay fixing the type of a variable being declared until after seeing the initializer, as this is required for unspecified-array-size, where the element type of the array is known but the number of elements (and thus the complete type of the array object) is only known after the entire initializer right-hand-

side has been seen. This feature therefore requires only a relatively minor change to existing machinery required for a conforming implementation.

There is no ABI or runtime impact. The feature is purely syntactic.

Bit-fields

The definition of bit-fields in C is underspecified, in that their types are only known if an lvalue expression of a bit-field additionally undergoes integer promotion. If no such promotion is performed, for example in a `_Generic` or comma expression, implementations diverge in their interpretation of the standard. Some always produce one of the types `bool`, `signed int` or `unsigned int`, others produce some implementation defined types of that reflect the width of the bit-field. The latter are not integer types in the sense of the standard because they only have to convert under promotion and need not to have any other property of integers, and, usually don't have documented declaration syntax.

It is not the place of this proposal here to sort out this inconsistency between different interpretations of the standard. This proposal specifies the feature in terms of the type specifier produced by a `typeof` operator, and therefore does not need to separately call out this divergent behaviour or propose a solution separately from that feature.

Combined definitions and compatible types

The semantics of underspecified declarations become complicated if they contain definitions for several objects where the inferred type has to be consistent.

In revision 5 the choice was that inferred types have to be the same, only having compatible types is not sufficient. This is particularly important for integer types, where mixing different enumeration types would have an ambiguity which type is chosen.

This partially caused revision 5 to be rejected at the January 2022 WG14 meeting and therefore the proposal resolves this difficulty by eliminating the syntax that would allow for any ambiguity here. Declarations using inferred types must now form separate declaration statements in line with existing GNU C practice.

Combined definitions and variably modified types

Revision 5 included a consistency problem in that different types within the same definition could not be checked for consistent VM elements.

This inconsistency is removed by not supporting multiple declarations within a single statement, so there is no longer any constraint to satisfy.

Specifiers

The constraint against specifying `auto` in combination with static, register, etc. is relaxed. `auto` now has no effect if it is not used to infer a type.

We expect implementations to continue to warn as a matter of QoI. This does not break any existing conforming code.

Scope

Existing practice in GNU C is that the identifier being declared has scope beginning after the end of the full-declaration, as opposed to all other identifiers which enter scope at the end of their declarator.

Unlike in revision 5, the same identifier in an enclosing scope is not shadowed and the identifier can appear to refer to itself; however it refers to the identifier in the enclosing scope. We tighten this with an additional constraint; we would expect GNU C implementations to warn on this construct (therefore conforming) as a matter of QoI in any case.

Implementation Experience

`__auto_type` is implemented by most (all?) compilers implementing the GNU C dialect or aiming for GCC compatibility. A non-exhaustive list includes: GCC, Clang, Intel CC, Helix QAC, Klocwork, armCC. It is generally used to implement type-generic macros in library headers. It does not appear to be widely used by developers in application code but is heavily tested by virtue of its appearance in Standard header implementations.

Many compilers exist which borrow components from GCC or Clang, and therefore inherit this feature intentionally or unintentionally.

A more comprehensive feature exists in C++ since C++11, which is based on template type deduction rules and can therefore use `auto` to infer parts of a partially-specified type, such as specifying that a declaration creates a pointer or reference but not what it is a pointer or reference to. This is in near-universal use by millions of C++ developers every day.

Specifying a feature closer to the C++ specifier would require substantial original wording in the Standard since C does not include templates, which the C++ feature is defined in terms of. We hope that C++ will set user expectation to be able to write `auto * foo = ...` such that implementers are encouraged to establish practice for this form as well.

Proposed wording

Changes are proposed against the wording in C23 draft n2731. Bolded text is new text.

Modify 6.2.1 “Scopes of identifiers”, paragraph 7:

> Structure, union, and enumeration tags have scope that begins just after the appearance of the tag in a type specifier that declares the tag. Each enumeration constant has scope that begins just after the appearance of its defining enumerator in an enumerator list. **An ordinary identifier whose type is specified by a non-ignored `auto` specifier has scope that begins just after the completion of its declaration footnote).** Any other identifier has scope that begins just after the completion of its declarator.
>
> **footnote) therefore such an identifier is in scope following the terminating semicolon of the declaration.**

Add a forward reference:

Forward references: declarations (6.7), **`auto` specifier (6.7.3),** function calls (6.5.2.2), function definitions (6.9.1), identifiers (6.4.2), macro replacement (6.10.3), name spaces of identifiers (6.2.3), source file inclusion (6.10.2), statements and blocks (6.8).

Add a new rule, *auto-specifier*, to the declaration syntax in 6.7 “Declarations”:

declaration-specifiers:
storage-class-specifier declaration-specifiers opt
type-specifier declaration-specifiers opt
auto-specifier declaration-specifiers opt
type-qualifier declaration-specifiers opt
function-specifier declaration-specifiers opt
alignment-specifier declaration-specifiers opt

Add a new paragraph after 6.7 paragraph 4:

If a declaration infers a type from an initializer, it shall declare only one declarator.

Remove `auto` from 6.7.1 “Storage-class specifiers”:

storage-class-specifier:
`typedef`
`extern`
`static`
`_Thread_local`
`register`

Add a new section after 6.7.2:

6.7.3 Auto specifier

Syntax

auto-specifier:
`auto`

Constraints

When an `auto` specifier is used to infer the type of a declarator, the declaration shall include an expression initializer.

A declaration shall contain at most one `auto` specifier.

The initializer expression for a declaration whose type is specified by the `auto` specifier shall not include the same identifier as as subexpression.

Semantics

The `auto` specifier is a type specifier that specifies the value-converted type of the initializer expression provided for the declaration.

The type specified is the type that would be specified by `typeof ((0, expr))`, where *`expr`* is the initializer expression.

If an explicit type specifier is provided as per 6.7.2, the `auto` specifier is ignored.

NOTE: the auto specifier cannot infer an array or function type because value conversion never results in an array or function type. An array object cannot be used as an initializer expression without being converted to a pointer to its first element.

EXAMPLE 1 the inferred type is the type of the initializer expression after value-conversion. Therefore it does not include qualifiers if the name of an object is specified:

```
void foo (int x, int const y) {
  auto a = x;
  auto b = y;

  int * pa = &a;
  int * pb = &b; // OK, b has type int, not int const
}

void bar (int x, int const y) {
  typeof (x) a = x;
  typeof (y) b = y;
  typeof ((0, y)) c = y; // remove_quals not

  int * pa = &a;
  int * pb = &b; // not OK, qualifier discarded
  int * pc = &c; // OK because c was value-converted
}
```

EXAMPLE 2 the `auto` specifier may be combined with storage class specifiers, but not used in declarations of more than one declarator:

```
void baz (int x, int const y) {
  static auto a = 1ul; // OK, inferred type is unsigned long

  auto b = 0, c = 1.0; // constraint violation

  int * pa = &a; // not OK, incompatible with unsigned long *
}
```

EXAMPLE 3 an array type cannot be inferred because an expression does not have array type after value-conversion:

```
void boo (void) {
  int arr[10];

  auto a = arr;
  _Static_assert (sizeof (a) == sizeof (int *)
      , "inferred type is value-converted");

  typeof (arr) b = arr; // error, invalid initializer
  typeof ((0, arr)) c = arr; // pointer

  _Static_assert (sizeof a == sizeof c
      , "inferred type is value-converted")
}
```

EXAMPLE 4 an identifier declared with an inferred type has scope that begins after the end of the complete declaration, rather than after the end of the declarator:

```
void qux (int x, int y) {
```

```
  int c = (c, 1); // non-inferred; in scope for the =

  auto a = 2 + x; // OK, doesn't refer to a
  auto b = (b, x); // error, b is not in scope

  {
    auto y = y; // constraint violation, would refer to outer y
  }
}
```

Recommended Practice

Implementations are encouraged to issue a diagnostic message when a declaration includes an `auto` specifier and an explicit type specifier that would cause the `auto` specifier to be ignored.

Forward references: `typeof` (?.?.?)

No additional text is needed to specify the combination of `auto` with `extern`. An auto-specifier is already a kind of type specifier and the existing wording about the types having to match is therefore sufficient.

References

- C23 n2731
- n2890 "Improve type generic programming"
- n2927 "Not-so-magic: typeof"
- C++11 n3337
- GCC typeof and __auto_type

This document is pared down from an original by Jens Gustedt. It builds heavily on JeanHeyd Meneide's accepted `typeof` feature.